# HydroGraphNet Original Code Archive

This document is an archival copy of the original HydroGraphNet source files before any transfer learning or domain adaptation modifications.

## Files included:

# train.py

*Original archived source file: train.py*

```
# SPDX-FileCopyrightText: Copyright (c) 2023 - 2025 NVIDIA CORPORATION & AFFILIATES.
# SPDX-License-Identifier: Apache-2.0

import time

import hydra
import torch
import torch.nn as nn
import torch_geometric as pyg
import wandb

from hydra.utils import to_absolute_path
from omegaconf import DictConfig

from torch_geometric.loader import DataLoader as PyGDataLoader

from torch.amp import GradScaler, autocast
from torch.nn.parallel import DistributedDataParallel
from torch.utils.data.distributed import DistributedSampler

from tqdm import tqdm  # ■ ADDED

from physicsnemo.datapipes.gnn.hydrographnet_dataset import HydroGraphDataset
from physicsnemo.distributed.manager import DistributedManager
from physicsnemo.utils.logging import PythonLogger, RankZeroLoggingWrapper
from physicsnemo.utils.logging.wandb import initialize_wandb
from physicsnemo.utils import load_checkpoint, save_checkpoint
from physicsnemo.models.meshgraphnet.meshgraphkan import MeshGraphKAN
from utils import compute_physics_loss


def collate_fn(batch):
    if isinstance(batch[0], tuple):
        graphs, physics_list = zip(*batch)
        batched_graph = pyg.data.from_data_list(graphs)
        physics_data = {}
        for key in physics_list[0].keys():
            physics_data[key] = torch.tensor(
                [d[key] for d in physics_list], dtype=torch.float
            )
        return batched_graph, physics_data
    else:
        return pyg.data.from_data_list(batch)


class MGNTrainer:
    def __init__(self, cfg: DictConfig, rank_zero_logger: RankZeroLoggingWrapper):
        assert DistributedManager.is_initialized()
        self.dist = DistributedManager()
        self.amp = cfg.amp
        self.noise_type = cfg.noise_type

        self.use_physics_loss = cfg.get("use_physics_loss", False)
        self.delta_t = cfg.get("delta_t", 1200.0)
        self.physics_loss_weight = cfg.get("physics_loss_weight", 1.0)

        mlp_act = "relu"
        if cfg.recompute_activation:
            mlp_act = "silu"

        dataset = HydroGraphDataset(
            name="hydrograph_dataset",
            data_dir=cfg.data_dir,
```

```
            prefix="M80",
            num_samples=500,
            n_time_steps=cfg.n_time_steps,
            k=4,
            noise_type=cfg.noise_type,
            noise_std=0.01,
            hydrograph_ids_file="train.txt",
            split="train",
            return_physics=self.use_physics_loss,
        )

        sampler = DistributedSampler(
            dataset,
            shuffle=True,
            drop_last=True,
            num_replicas=self.dist.world_size,
            rank=self.dist.rank,
        )

        self.dataloader = PyGDataLoader(
            dataset,
            batch_size=cfg.batch_size,
            sampler=sampler,
            pin_memory=True,
            num_workers=cfg.num_dataloader_workers,
            collate_fn=collate_fn,
        )

        self.model = MeshGraphKAN(
            cfg.num_input_features,
            cfg.num_edge_features,
            cfg.num_output_features,
            mlp_activation_fn=mlp_act,
            do_concat_trick=cfg.do_concat_trick,
            num_processor_checkpoint_segments=cfg.num_processor_checkpoint_segments,
            recompute_activation=cfg.recompute_activation,
        ).to(self.dist.device)

        if self.dist.world_size > 1:
            self.model = DistributedDataParallel(
                self.model,
                device_ids=[self.dist.local_rank],
                output_device=self.dist.device,
                broadcast_buffers=self.dist.broadcast_buffers,
                find_unused_parameters=self.dist.find_unused_parameters,
            )

        self.model.train()
        self.criterion = nn.MSELoss()
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=cfg.lr)

        self.scheduler = torch.optim.lr_scheduler.LambdaLR(
            self.optimizer, lr_lambda=lambda epoch: cfg.lr_decay_rate**epoch
        )

        self.scaler = GradScaler()

        self.epoch_init = load_checkpoint(
            to_absolute_path(cfg.ckpt_path),
            models=self.model,
            optimizer=self.optimizer,
            scheduler=self.scheduler,
            scaler=self.scaler,
            device=self.dist.device,
        )

    def train(self, batch):
        if self.use_physics_loss:
            graph, physics_data = batch
        else:
            graph = batch
            physics_data = None

        graph = graph.to(self.dist.device)

        # >>> ADDED: Strict NaN scrubbing for graph tensors to prevent poisoning from faulty standar
        graph.x = torch.nan_to_num(graph.x, nan=0.0, posinf=0.0, neginf=0.0)
```

```
            graph.y = torch.nan_to_num(graph.y, nan=0.0, posinf=0.0, neginf=0.0)
            if hasattr(graph, 'edge_attr') and graph.edge_attr is not None:
                graph.edge_attr = torch.nan_to_num(graph.edge_attr, nan=0.0, posinf=0.0, neginf=0.0)

            if physics_data is not None:
                physics_data = {k: v.to(self.dist.device) for k, v in physics_data.items()}
                # >>> ADDED: Sanitize physics data tensors
                physics_data = {k: torch.nan_to_num(v, nan=0.0, posinf=0.0, neginf=0.0) for k, v in physi

            self.optimizer.zero_grad()
            loss, loss_dict = self.forward(graph, physics_data)
            self.backward(loss)
            self.scheduler.step()
            return loss, loss_dict

        def forward(self, graph, physics_data):
            with autocast(device_type=self.dist.device.type, enabled=self.amp):
                pred = self.model(graph.x, graph.edge_attr, graph)

                # >>> ADDED: Clamp predictions to prevent FP16 numerical explosions during early unstable
                pred = torch.nan_to_num(pred, nan=0.0, posinf=0.0, neginf=0.0)
                pred = torch.clamp(pred, min=-10.0, max=10.0)

                mse_loss = self.criterion(pred, graph.y)
                loss = mse_loss
                loss_dict = {"total_loss": loss, "mse_loss": mse_loss}

                if self.use_physics_loss and physics_data is not None:
                    phy_loss = compute_physics_loss(
                        pred, physics_data, graph, delta_t=self.delta_t
                    )
                    loss = loss + self.physics_loss_weight * phy_loss
                    loss_dict["physics_loss"] = phy_loss

            return loss, loss_dict

        def backward(self, loss):
            if self.amp:
                self.scaler.scale(loss).backward()
                self.scaler.step(self.optimizer)
                self.scaler.update()
            else:
                loss.backward()
                self.optimizer.step()


    @hydra.main(version_base="1.3", config_path="conf", config_name="config")
    def main(cfg: DictConfig) -> None:
        DistributedManager.initialize()
        dist = DistributedManager()

        initialize_wandb(
            project="Modulus-Launch",
            entity="Modulus",
            name="Vortex_Shedding-Training",
            group="Vortex_Shedding-DDP-Group",
            mode=cfg.wandb_mode,
        )

        logger = PythonLogger("main")
        rank_zero_logger = RankZeroLoggingWrapper(logger, dist)
        rank_zero_logger.file_logging()

        trainer = MGNTrainer(cfg, rank_zero_logger)

        for epoch in tqdm(
            range(trainer.epoch_init, cfg.epochs),
            desc="Epochs",
            disable=dist.rank != 0,
        ):
            epoch_loss = 0.0
            num_batches = 0

            dataloader_tqdm = tqdm(
                trainer.dataloader,
                desc=f"Epoch {epoch}",
                leave=False,
```

```
            disable=dist.rank != 0,
        )

        for batch in dataloader_tqdm:
            loss, loss_dict = trainer.train(batch)

            batch_loss = loss.detach().item()
            epoch_loss += batch_loss
            num_batches += 1

            if torch.cuda.is_available():
                mem_alloc = torch.cuda.memory_allocated() / 1024**3
                mem_reserved = torch.cuda.memory_reserved() / 1024**3
            else:
                mem_alloc = 0.0
                mem_reserved = 0.0

            dataloader_tqdm.set_postfix(
                loss=f"{batch_loss:.4e}",
                gpu_mem=f"{mem_alloc:.2f}/{mem_reserved:.2f} GB",
            )

        avg_loss = epoch_loss / num_batches
        rank_zero_logger.info(f"Epoch {epoch} | Avg Loss: {avg_loss:.4e}")

        if dist.rank == 0:
            save_checkpoint(
                to_absolute_path(cfg.ckpt_path),
                models=trainer.model,
                optimizer=trainer.optimizer,
                scheduler=trainer.scheduler,
                scaler=trainer.scaler,
                epoch=epoch,
            )


if __name__ == "__main__":
    main()
```

# inference.py

*Original archived source file: inference.py*

```
# SPDX-FileCopyrightText: Copyright (c) 2023 - 2025 NVIDIA CORPORATION & AFFILIATES.
# SPDX-FileCopyrightText: All rights reserved.
# SPDX-License-Identifier: Apache-2.0

import os
import json
import torch
import hydra
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from omegaconf import DictConfig, OmegaConf
from hydra.utils import to_absolute_path

from physicsnemo.utils import load_checkpoint
from physicsnemo.datapipes.gnn.hydrographnet_dataset import HydroGraphDataset
from physicsnemo.models.meshgraphnet.meshgraphkan import MeshGraphKAN
from torch_geometric.utils import to_networkx
from torch import amp

from tqdm import tqdm
import psutil
import numpy as np

# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
# Delaunay triangulation (OPTION 1 – recommended)
# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
from scipy.spatial import Delaunay


def triangulate_nodes_xy(nodes_xy):
    """
    Perform Delaunay triangulation on node (x,y) positions.
    Returns triangle indices suitable for Unreal Engine.
    """
    tri = Delaunay(nodes_xy)
    return tri.simplices.tolist()


def get_system_stats(device):
    cpu = psutil.cpu_percent()
    mem = psutil.virtual_memory().percent

    if device.type == "cuda":
        gpu_mem = torch.cuda.memory_allocated() / 1024**3
        gpu_mem_max = torch.cuda.max_memory_allocated() / 1024**3
        return {
            "CPU%": f"{cpu:.1f}",
            "RAM%": f"{mem:.1f}",
            "GPU_GB": f"{gpu_mem:.2f}/{gpu_mem_max:.2f}",
        }
    else:
        return {
            "CPU%": f"{cpu:.1f}",
            "RAM%": f"{mem:.1f}",
        }


def describe_performance(nrmse):
    if nrmse <= 0.02:
        return "Excellent"
    elif nrmse <= 0.04:
        return "Very Good"
    elif nrmse <= 0.06:
        return "Good"
    elif nrmse <= 0.08:
        return "Tolerable"
    else:
        return "Poor"


def create_animation(
    rollout_predictions,
```

```
    ground_truth,
    initial_graph,
    rmse_list,
    output_path,
    global_anim_pbar,
    time_per_step=20 / 60,
):
    plt.rcParams["font.family"] = "serif"
    plt.rcParams["font.serif"] = ["Times New Roman", "DejaVu Serif", "Liberation Serif"]
    plt.rcParams["font.size"] = 20

    fig, axes = plt.subplots(2, 2, figsize=(30, 30))
    cax1 = fig.add_axes([0.05, 0.53, 0.02, 0.35])
    cax2 = fig.add_axes([0.95, 0.53, 0.02, 0.35])
    cax3 = fig.add_axes([0.05, 0.1, 0.02, 0.35])

    init_node_feats = initial_graph.x
    pos = {
        i: (init_node_feats[i, 0].item(), init_node_feats[i, 1].item())
        for i in range(init_node_feats.shape[0])
    }

    all_vals = torch.cat(rollout_predictions + ground_truth)
    vmin_global = all_vals.min().item()
    vmax_global = all_vals.max().item()

    def update(frame):
        for ax in axes.flat:
            ax.clear()

        current_time = (frame + 1) * time_per_step
        g_nx = to_networkx(initial_graph).to_undirected()

        pred_vals = rollout_predictions[frame].cpu().numpy()
        nodes = nx.draw_networkx_nodes(
            g_nx, pos, node_color=pred_vals, cmap=plt.cm.viridis,
            ax=axes[0, 0], vmin=vmin_global, vmax=vmax_global, node_size=250
        )
        nx.draw_networkx_edges(g_nx, pos, alpha=0.5, ax=axes[0, 0])
        axes[0, 0].set_title(f"Time {current_time:.2f} Hours - Prediction")
        fig.colorbar(nodes, cax=cax1)

        gt_vals = ground_truth[frame].cpu().numpy()
        nodes = nx.draw_networkx_nodes(
            g_nx, pos, node_color=gt_vals, cmap=plt.cm.viridis,
            ax=axes[0, 1], vmin=vmin_global, vmax=vmax_global, node_size=250
        )
        nx.draw_networkx_edges(g_nx, pos, alpha=0.5, ax=axes[0, 1])
        axes[0, 1].set_title("Ground Truth")
        fig.colorbar(nodes, cax=cax2)

        abs_vals = torch.abs(
            rollout_predictions[frame] - ground_truth[frame]
        ).cpu().numpy()
        nodes = nx.draw_networkx_nodes(
            g_nx, pos, node_color=abs_vals, cmap=plt.cm.viridis,
            ax=axes[1, 0], vmin=vmin_global, vmax=vmax_global, node_size=250
        )
        nx.draw_networkx_edges(g_nx, pos, alpha=0.5, ax=axes[1, 0])
        axes[1, 0].set_title("Absolute Error")
        fig.colorbar(nodes, cax=cax3)

        times = [(i + 1) * time_per_step for i in range(frame + 1)]
        axes[1, 1].plot(times, rmse_list[: frame + 1], linewidth=3)
        axes[1, 1].set_title("RMSE Over Time")
        axes[1, 1].grid(True)

        global_anim_pbar.update(1)

    ani = animation.FuncAnimation(
        fig,
        update,
        frames=len(rollout_predictions),
        repeat=False,
    )
    ani.save(output_path, writer="pillow", fps=2)
    plt.close(fig)
```

```
@hydra.main(version_base="1.3", config_path="conf", config_name="config")
def main(cfg: DictConfig):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    ENABLE_ANIMATION = cfg.anim

    print("Configuration:\n", OmegaConf.to_yaml(cfg))

    test_dataset = HydroGraphDataset(
        data_dir=cfg.test_dir,
        prefix=cfg.get("prefix", "M80"),
        n_time_steps=cfg.n_time_steps,
        hydrograph_ids_file=cfg.get("test_ids_file", "test.txt"),
        split="test",
        rollout_length=cfg.num_test_time_steps,
        return_physics=False,
    )

    model = MeshGraphKAN(
        cfg.num_input_features,
        cfg.num_edge_features,
        cfg.num_output_features,
    ).to(device)

    load_checkpoint(to_absolute_path(cfg.ckpt_path), models=model, device=device)
    model.eval()

    # ==================== ADDITION (FIX #2, #3, #5) ====================
    unreal_export = {
        "dt": float(cfg.delta_t),
        "coordinate_system": "HydroGraphNet",
        "scale_hint": "1 unit = 1 meter",

        "unreal_transform": {
            "axis_mapping": "X=x, Y=y, Z=z",
            "up_axis": "Z",
            "unit_scale_cm": 100.0
        },

        "terrain": {
            "unit": "meters",
            "z_scale": 1.0,
            "vertical_datum": "model_relative"
        },

        "binary_payload": {
            "enabled": False,
            "recommended_layout": [
                "water_depth.bin [T,N]",
                "water_surface_z.bin [T,N]"
            ]
        },

        "graphs": []
    }
    # ====================================================================

    global_anim_pbar = None
    if ENABLE_ANIMATION:
        global_anim_pbar = tqdm(
            total=len(test_dataset) * cfg.num_test_time_steps,
            desc="Creating Animation",
            unit="frame",
            dynamic_ncols=True,
        )

    test_pbar = tqdm(
        range(len(test_dataset)),
        desc="Testing Samples",
        unit="graph",
        dynamic_ncols=True,
    )

    all_metrics = []  # >>> ADDED: Collect metrics for table printing

    for idx in test_pbar:
        test_pbar.set_postfix(get_system_stats(device))
        g, rollout_data = test_dataset[idx]
```

```
g = g.to(device).clone()

# >>> ADDED: Sanitize inputs immediately to prevent NaN propagation from old/bad datasets
g.x = torch.nan_to_num(g.x, nan=0.0, posinf=0.0, neginf=0.0)
if hasattr(g, 'edge_attr') and g.edge_attr is not None:
    g.edge_attr = torch.nan_to_num(g.edge_attr, nan=0.0, posinf=0.0, neginf=0.0)

nodes_xy = g.x[:, :2].cpu().numpy()
triangles = triangulate_nodes_xy(nodes_xy)

# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
# ADDITION: Generate OBJ file with 100% UNMODIFIED ORIGINAL COORDINATES
# NO TRANSLATION, NO CENTERING, NO SCALING - EXACT DATASET VALUES
# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
obj_output_dir = os.path.join("outputs", "obj")
os.makedirs(obj_output_dir, exist_ok=True)
obj_filepath = os.path.join(obj_output_dir, f"mesh_{idx}.obj")

# Calculate bounds for user guidance (does NOT modify mesh data)
xmin, xmax = g.x[:, 0].min().item(), g.x[:, 0].max().item()
ymin, ymax = g.x[:, 1].min().item(), g.x[:, 1].max().item()
zmin, zmax = g.x[:, 2].min().item(), g.x[:, 2].max().item()

with open(obj_filepath, 'w') as f_obj:
    # EXTREMELY CLEAR HEADER: Emphasize NO MODIFICATION to coordinates
    f_obj.write("# HYDROGRAPHNET TERRAIN MESH - ABSOLUTELY UNMODIFIED COORDINATES\n")
    f_obj.write("# WARNING: Mesh uses ORIGINAL DATASET COORDINATES (may be large UTM values)\
    f_obj.write("#          If mesh appears 'invisible' in 3D software:\n")
    f_obj.write("#          1. Press 'F' (Frame Selected) or 'Home' (View All) in Blender\n"
    f_obj.write("#          2. Or manually zoom out EXTREMELY far in viewport\n")
    f_obj.write("#          3. Coordinates are in METERS - no scaling applied\n")
    f_obj.write(f"# COORDINATE BOUNDS (meters): X[{xmin:.2f}, {xmax:.2f}] Y[{ymin:.2f}, {yma
    f_obj.write("# Vertex format: v x y z  (direct from dataset column 0=x, 1=y, 2=bed_eleva
    f_obj.write("# Face format: f v1 v2 v3 (1-based indexing)\n")
    f_obj.write("# 1 unit = 1 real-world meter - NO TRANSFORMATIONS APPLIED\n\n")

    # CRITICAL: Write vertices with ABSOLUTELY ORIGINAL VALUES - ZERO MODIFICATION
    for node_idx in range(g.num_nodes):
        # Direct passthrough - NO arithmetic operations on coordinates
        x = g.x[node_idx, 0].item()  # Raw x from dataset
        y = g.x[node_idx, 1].item()  # Raw y from dataset
        z = g.x[node_idx, 2].item()  # Raw bed elevation from dataset
        f_obj.write(f"v {x:.6f} {y:.6f} {z:.6f}\n")

    f_obj.write("\n# Faces (Delaunay triangulation)\n")
    for tri in triangles:
        v1, v2, v3 = tri[0] + 1, tri[1] + 1, tri[2] + 1
        f_obj.write(f"f {v1} {v2} {v3}\n")
# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
# END OF ADDITION - ALL ORIGINAL CODE BELOW REMAINS UNCHANGED
# >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>

X_iter = g.x.to(device).clone()
rollout_preds, gt_list, rmse_list = [], [], []

wd_gt_seq = rollout_data["water_depth_gt"].to(device)

# >>> ADDED: Sanitize ground truth to prevent downstream metric NaNs
wd_gt_seq = torch.nan_to_num(wd_gt_seq, nan=0.0, posinf=0.0, neginf=0.0)

for t in range(cfg.num_test_time_steps):
    static = X_iter[:, :12]
    wd = X_iter[:, 12 : 12 + cfg.n_time_steps]
    vol = X_iter[:, 12 + cfg.n_time_steps : 12 + 2 * cfg.n_time_steps]

    X_input = torch.cat([static, wd, vol], dim=1)
    with torch.no_grad(), amp.autocast("cuda", dtype=torch.float16):
        pred = model(X_input, g.edge_attr.to(device), g)

    # >>> ADDED: Sanitize FP16 autocast overflows so NaNs don't cascade into subsequent times
    pred = torch.nan_to_num(pred, nan=0.0, posinf=0.0, neginf=0.0)

    new_wd = wd[:, -1:] + pred[:, 0:1]
    wd = torch.cat([wd[:, 1:], new_wd], dim=1)

    rollout_preds.append(new_wd.squeeze(1).cpu())
    gt_list.append(wd_gt_seq[t].cpu())
```

```
    rmse_list.append(
        torch.sqrt(torch.mean((new_wd.squeeze(1) - wd_gt_seq[t]) ** 2)).item()
    )

    X_iter = torch.cat([static, wd, vol], dim=1)

# ====================== ADDITION (METRICS FIX) ======================
preds_all = torch.cat(rollout_preds).numpy()
gt_all = torch.cat(gt_list).numpy()

# >>> ADDED: Final sanitization pass before numpy operations
preds_all = np.nan_to_num(preds_all, nan=0.0, posinf=0.0, neginf=0.0)
gt_all = np.nan_to_num(gt_all, nan=0.0, posinf=0.0, neginf=0.0)

rmse = float(np.sqrt(np.mean((preds_all - gt_all) ** 2)))
mae = float(np.mean(np.abs(preds_all - gt_all)))
# MAPE REMOVED: Unreliable for flood depth (division by near-zero values in dry cells)
nrmse = float(rmse / (np.max(gt_all) - np.min(gt_all) + 1e-6))
bias = float(np.mean(preds_all - gt_all))

ss_res = np.sum((gt_all - preds_all) ** 2)
ss_tot = np.sum((gt_all - np.mean(gt_all)) ** 2)
r2 = float(1 - ss_res / (ss_tot + 1e-6))
nse = r2

r = np.corrcoef(preds_all.flatten(), gt_all.flatten())[0, 1]
# >>> ADDED: Handle case where ground truth has 0 variance (e.g. perfectly dry map) causing N
if np.isnan(r):
    r = 0.0

alpha = np.std(preds_all) / (np.std(gt_all) + 1e-6)
beta = np.mean(preds_all) / (np.mean(gt_all) + 1e-6)
kge = float(1 - np.sqrt((r - 1) ** 2 + (alpha - 1) ** 2 + (beta - 1) ** 2))
# ====================================================================

# ====================== ADDITION (FIX #1) ======================
bed = g.x[:, 2].cpu().numpy()
water_depth = [p.numpy().tolist() for p in rollout_preds]
water_surface_z = [(bed + p.numpy()).tolist() for p in rollout_preds]
# ================================================================

metrics_entry = {  # >>> ADDED: Store metrics for table
    "rmse": rmse,
    "mae": mae,
    # MAPE REMOVED from metrics dictionary
    "nrmse": nrmse,
    "bias": bias,
    "r2": r2,
    "nse": nse,
    "kge": kge,
    "score": describe_performance(nrmse)
}
all_metrics.append(metrics_entry)  # >>> ADDED: Append to collection

unreal_export["graphs"].append({
    "graph_id": idx,
    "num_nodes": int(g.num_nodes),
    "num_timesteps": int(cfg.num_test_time_steps),
    "nodes_xyz": g.x[:, :3].cpu().numpy().tolist(),
    "bed_elevation": g.x[:, 2].cpu().numpy().tolist(),
    "edges": g.edge_index.cpu().numpy().T.tolist(),
    "triangles": triangles,

    "water_elevation": [p.tolist() for p in rollout_preds],

    "water_depth": water_depth,
    "water_surface_z": water_surface_z,

    "velocity": [[[0.0, 0.0] for _ in range(g.num_nodes)]
                 for _ in range(cfg.num_test_time_steps)],
    "velocity_valid": False,

    "bounds": {
        "xmin": float(g.x[:, 0].min()),
        "xmax": float(g.x[:, 0].max()),
        "ymin": float(g.x[:, 1].min()),
        "ymax": float(g.x[:, 1].max())
```

```
            },
            "metrics": metrics_entry  # >>> ADDED: Include in export too (without MAPE)
        })

        if ENABLE_ANIMATION:
            os.makedirs("animations", exist_ok=True)
            create_animation(
                rollout_preds,
                gt_list,
                g,
                rmse_list,
                f"animations/animation_{idx}.gif",
                global_anim_pbar,
            )

    if ENABLE_ANIMATION:
        global_anim_pbar.close()

    os.makedirs("outputs", exist_ok=True)
    with open("outputs/anim_data_op.json", "w") as f:
        json.dump(unreal_export, f, indent=2)

    # >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
    # PRINT METRICS TABLE (MAPE COLUMN REMOVED)
    # >>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
    print("\n" + "=" * 120)
    print(f"{'Graph':<6} {'RMSE':>8} {'MAE':>8} {'NRMSE':>7} {'Bias':>8} {'R²':>7} {'NSE':>7} {'KGE'
    print("=" * 120)
    for i, m in enumerate(all_metrics):
        print(f"{i:<6} {m['rmse']:>8.4f} {m['mae']:>8.4f} {m['nrmse']:>7.4f} {m['bias']:>8.4f} "
              f"{m['r2']:>7.4f} {m['nse']:>7.4f} {m['kge']:>7.4f} {m['score']:<12}")
    print("=" * 120)

    print("\nSaved Unreal Engine animation data to: ./outputs/anim_data_op.json")
    print(f"Saved 3D terrain meshes (100% UNMODIFIED coordinates) to: ./{obj_output_dir}/mesh_*.obj"
    print("\n■■  IMPORTANT: Meshes use ORIGINAL DATASET COORDINATES (may be large UTM values)")
    print("   To see the mesh in 3D software:")
    print("   • Blender: Press 'Home' key or 'View → Frame All'")
    print("   • Maya: Press 'F' after selecting nothing (frames entire scene)")
    print("   • 3ds Max: Press 'Ctrl+Shift+Z' (Zoom Extents All)")
    print("   • Coordinates are in METERS - no scaling/translation applied")
    print("■ OBJ files contain EXACT dataset values - ready for geospatial workflows")
    print("■ Ready for flood visualization in Unreal Engine (precomputed playback)")


if __name__ == "__main__":
    main()
```

# config.yaml

*Original archived source file: config.yaml*

```
# SPDX-FileCopyrightText: Copyright (c) 2023 - 2025 NVIDIA CORPORATION & AFFILIATES.
# SPDX-FileCopyrightText: All rights reserved.
# SPDX-License-Identifier: Apache-2.0
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

# Configuration file for HydroGraphNet training.
# This file is used by Hydra to configure the training run.

hydra:
  job:
    chdir: False  # Change directory to the job's working directory.
  run:
    dir: ./outputs_phy/  # Directory to save outputs.

# Data configuration: paths for training and testing datasets.
data_dir: ./HGN_train
test_dir: ./HGN_test

# Animation output
anim: true

# Training configuration.
batch_size: 8
epochs: 120
num_training_samples: 400
num_training_time_steps: 300
lr: 0.0001
lr_decay_rate: 0.9999979
weight_decay: 0.0001
num_input_features: 16    # Number of node input features.
num_output_features: 2    # Number of output features (e.g., depth and volume differences).
num_edge_features: 3      # Number of edge features.

# Noise settings.
noise_type: "none"        # Options: "none", "pushforward", "only_last", "correlated", etc.
n_time_steps: 2           # Number of time steps in the sliding window.

# Physics loss settings.
use_physics_loss: true
delta_t: 1200.0
physics_loss_weight: 1.0

# Performance and optimization configurations.
use_apex: True            # Use NVIDIA Apex for mixed precision if available.
amp: False                # Automatic mixed precision flag.
jit: False                # Use TorchScript JIT compilation.
num_dataloader_workers: 4
do_concat_trick: False
num_processor_checkpoint_segments: 0
recompute_activation: False

# WandB logging configuration.
wandb_mode: disabled
watch_model: False

# Checkpoint path.
ckpt_path: "./checkpoints_phy"

# Test and visualization configuration.
num_test_samples: 10
num_test_time_steps: 30
```